

A Comparative Analysis of Phylogenetic Tree Manipulation Tools: Benchmarking Tree Traversal

Emma Gairaud Hess

June 1, 2026

Abstract

Phylogenetic tree manipulation is essential to evolutionary biology and epidemiology. This study presents a survey and benchmarking of 10 phylogenetic packages and tools, evaluating their performance on tree traversal operations (preorder, postorder, level-order) and ultrametricity checking. The tests were conducted on randomly generated trees of 1,000 leaves. The performance metrics were measured in R-based (*ape*, *castor*), Python-based (*dendropy*, *cogent3*, *Biopython*, *ete3*, *toytree*), Julia-based (*Phylo.jl*), Go-based (*gotree*) and Rust-based (*phylo-rs*) tools.

Keywords: phylogenetics, tree traversal, benchmarking,

Contents

1	Introduction	3
2	Methods	3
2.1	Part I: Tree Traversal Benchmarking	3
2.1.1	Benchmark Design	3
2.1.2	Tree Generation Procedure	3
2.1.3	Benchmarking Procedure	3
2.1.4	Tools Evaluated	3
2.1.5	Statistical Analysis	4
2.2	Part II: Birth-Death Model Benchmarking	4
2.2.1	Outline	4
2.2.2	Proposed Benchmarking Framework	5
3	Results and Discussion	5
3.1	Tree Traversal Performance	5
3.1.1	Overview	5
3.1.2	Compiled vs. Interpreted Implementations	5
3.1.3	Tree Size Scaling	6
3.2	Birth-Death Model Inference	6
3.2.1	Parameter Estimation Accuracy	6
3.2.2	Computational Efficiency at Scale	6
3.2.3	Sampling Bias Effects	6
4	Conclusions and Perspectives	6
4.1	Key Findings	6
4.2	Recommendations	6
4.3	Future Directions	7
5	References	7
A	Detailed Benchmark Results	8
A.1	Tool Specifications	8
A.2	Performance Plots	8
A.3	Raw Benchmark Data Tables	8

1 Introduction

Phylogenetic trees are data structures used in evolutionary biology, representing the evolutionary relationships between taxa. Modern phylogenetic studies frequently involve large trees (thousands to millions of leaves), which requires efficient algorithms to manipulate said trees.

Current phylogenetic software is remarkably diverse, with implementations available across multiple programming languages. While many studies focus on phylogenetic inference accuracy, comparative performance evaluation of fundamental tree operations is limited.

This report addresses two key objectives:

1. Benchmarking basic tree operations across several phylogenetic packages

2 Methods

2.1 Part I: Tree Traversal Benchmarking

2.1.1 Benchmark Design

A standardized benchmarking framework was implemented to evaluate four core operations:

- **Preorder traversal:** Root-to-leaf depth-first search (DFS) (fig.1A)
- **Postorder traversal:** Leaf-to-root DFS (fig.1B)
- **Level-order traversal:** Queue-based breadth-first search (BFS) (fig.1C)
- **Ultrametricity checking:** Verifies that all leaves are equidistant from the root (fig.2)

2.1.2 Tree Generation Procedure

Test trees were generated in *ape* using coalescent models of 1,000 and 10,000 leaves:

- Ultrametric trees: `rcoal(n)` generates a coalescent tree of n leaves
- Heterochronic trees: `rtree(n)` generates a general tree of n leaves

For each tree type and size, 1,000 trees were generated and stored in *nwk* files.

2.1.3 Benchmarking Procedure

Timing measurements followed a warmup iteration to account for JIT compilation and memory allocation overhead. Execution times were recorded in microseconds.

2.1.4 Tools Evaluated

ape (R package)

Framework: *ape* is the foundational R package for phylogenetic analysis. It provides native tree data structures using edge lists stored in $n \times 2$ matrices. Tree operations are implemented in compiled C code for performance. The package includes native R plotting via base graphics. Benchmarks use the `is.ultrametric()` function from the compiled backend.

castor (R package)

Framework: [TBD] Specialized R package for tree inference and analysis. To be documented.

dendropy (Python)

Framework: Pure Python implementation of phylogenetic data structures. *dendropy* uses object-oriented design with Node and Tree classes. Traversal algorithms are implemented in Python, relying on recursive depth-first and iterative breadth-first approaches. No compiled backend; performance bottlenecked by Python interpreter overhead.

cogent3 (Python)

Framework: *cogent3* is designed for comparative genomics with phylogenetic capabilities. Implementation strategy prioritizes code clarity over raw speed. Includes optional Cython compilation for performance-critical sections. Tree operations leverage NumPy-based array operations where applicable.

Biopython (Python)

Framework: General-purpose bioinformatics package with phylogenetic submodule (`Bio.Phylo`). Provides wrappers around external phylogenetic tools while offering native Python tree structures. Implementation emphasizes compatibility with diverse tree formats (Newick, Nexus, PhyloXML) over computational efficiency.

ete3 (Python)

Framework: Pure Python implementation optimized for interactive phylogenetic analysis. *ete3* uses recursive tree structures with lazy evaluation of some properties. Recent versions include compiled extensions for critical operations. The library emphasizes usability and visualization (including ASCII display) alongside performance.

toytree (Python)

Framework: Lightweight Python tree library designed for rapid phylogenetic analysis. Implements core operations in compiled C via Cython bindings, providing significant speedup over pure Python alternatives. Integrates with Jupyter notebooks for interactive workflows.

Phylo.jl (Julia)

Framework: Julia package providing multiple dispatch-based tree implementations. Julia's LLVM JIT compilation enables performance comparable to compiled languages. Tree operations are implemented in Julia with selective use of optimized algorithms. Notable: substantial variance in performance indicates JIT warm-up effects across iterations.

2.1.5 Statistical Analysis

For each tool and operation, summary statistics were computed:

- Mean, median, and standard deviation of execution times
- Min/max times to identify outliers and JIT compilation effects

Summary visualizations (violin plots, box plots) were generated for qualitative comparison.

2.2 Part II: Birth-Death Model Benchmarking

2.2.1 Outline

Birth-death processes model the transmission dynamics of epidemics, with birth rate λ (transmission) and death rate μ (removal/sampling). Given a phylogenetic tree of sampled individuals, inference of λ and μ estimates transmission rate and the proportion of infected individuals who were sampled.

Key challenges in benchmarking:

- Large datasets: Real outbreak trees frequently contain $> 100,000$ tips
- Sampling effects: Non-random sampling biases parameter estimates
- Computational scalability: Likelihood evaluation requires tree traversal and recursive computation

2.2.2 Proposed Benchmarking Framework

[TBD]

Benchmark specifications (to be documented):

- Birth-death tree generation with varying λ , μ , and sampling proportions
- Parameter inference via maximum likelihood estimation
- Comparison of algorithm efficiency across tools
- Assessment of accuracy/bias under different sampling regimes

3 Results and Discussion

3.1 Tree Traversal Performance

3.1.1 Overview

Benchmarking results across eight tools reveal substantial performance variation: spanning from ~ 50 microseconds (*cogent3* preorder on 1k-tree) to ~ 70 milliseconds (*ete3* ultrametric check on 1k-tree).

A summary comparison is provided in Table 1.

Table 1: Summary of tool performance rankings across operations. Times in microseconds (1k-leaf trees).

Tool	Preorder	Postorder	Level-order	Ultrametric
Cogent3	148	404	161	261
Phylo.jl	289	3,023	1,296	3,258
Toytree	2,086	1,858	1,790	5,756
ETE3	2,196	7,921	1,998	70,056
Dendropy	4,466	3,836	3,790	4,954
Biopython	28,054	28,477	7,360	3,526

3.1.2 Compiled vs. Interpreted Implementations

High-Performance Tools (*cogent3*, *toytree*) *cogent3* exhibits exceptional performance for traversal operations ($\sim 150 - 400\mu s$), likely due to algorithmic optimization or compiled extensions in critical sections. *toytree* also demonstrates strong performance ($\sim 1.8 - 2.1\mu s$ for traversal, $\sim 5.8\mu s$ for ultrametric check), consistent with its Cython-based implementation.

Pure Python Tools (*dendropy*, *Biopython*, *ete3*) *dendropy* achieves moderate performance ($\sim 3.8 - 4.5\mu s$), acceptable for interactive workflows but substantially slower than compiled alternatives. *Biopython* exhibits variable performance: excellent on ultrametricity checking ($\sim 3.5\mu s$) but slow on traversals ($\sim 28\mu s$). *ete3* shows dramatic overhead on ultrametricity checks ($\sim 70\mu s$), suggesting algorithmic differences or missing optimizations in the implementation.

Julia (Phylo.jl) *Phylo.jl* demonstrates moderate traversal performance ($\sim 289\mu s$ preorder) with high variance, reflecting JIT compilation warm-up costs. Postorder traversal is notably slower ($\sim 3\mu s$), indicating potential algorithmic inefficiency. The high variance (std. dev. $\sim 5100\mu s$) suggests that performance predictions are unreliable on first-run executions.

3.1.3 Tree Size Scaling

[TBD]

Evaluation on larger trees (10,000 leaves) to assess algorithmic complexity and scalability.

3.2 Birth-Death Model Inference

[TBD]

[Section to be completed with benchmarking results once framework is implemented]

3.2.1 Parameter Estimation Accuracy

[TBD]

3.2.2 Computational Efficiency at Scale

[TBD]

3.2.3 Sampling Bias Effects

[TBD]

4 Conclusions and Perspectives

4.1 Key Findings

1. **Substantial Performance Variation:** Performance differences exceed 400-fold across tools, emphasizing the importance of tool selection for computationally intensive phylogenetic analyses.
2. **Compiled Backends Essential:** Tools with compiled implementations (*cogent3*, *toytrees*, *Phylo.jl*JIT) outperform pure Python alternatives by 5–200 $\times$.
3. **Algorithm Choice Matters:** Identical operations show significant variance across implementations, suggesting different algorithmic approaches (e.g., *ete3*'s ultrametricity check).
4. **Language Trade-offs:** R-based tools (*ape*) balance performance with extensive statistical capabilities. Julia shows promise but current implementations exhibit high variance, likely reflecting JIT overhead.

4.2 Recommendations

- **Interactive analysis:** *ete3* or *toytrees* for their visualization and usability
- **Performance-critical applications:** *cogent3* or *toytrees* for maximum throughput
- **Large-scale inference:** Compiled solutions (*ape* with C backend) recommended
- **Future consideration:** Investigate Rust-based implementations (e.g., *shiftcarb*, *phylo-rs*)

4.3 Future Directions

1. Complete benchmarking of birth-death process inference across tools
2. Evaluate performance on real outbreak datasets (>100k leaves)
3. Investigate memory usage and scalability limits
4. Assess accuracy of parameter estimation under non-random sampling
5. Profile code hotspots to identify optimization opportunities
6. Explore hybrid approaches combining tools' strengths (e.g., Julia for inference, Python for visualization)

5 References

References

- [1] Paradis, E., Claude, J., & Strimmer, K. (2004). APE: analyses of phylogenetics and evolution in R language. *Bioinformatics*, 20(2), 289–290.
- [2] Sukumaran, J., & Holder, M. T. (2010). DendroPy: a Python library for phylogenetic computing. *Bioinformatics*, 26(12), 1569–1571.
- [3] Knight, R., & others (2018). Cogent3: a toolkit for machine learning on biological sequences. *Molecular Biology and Evolution*.
- [4] Cock, P. J., Chang, J. T., Chang, K. M., & others (2009). Biopython: freely available Python tools for computational biology and bioinformatics. *Bioinformatics*, 25(3), 422–423.
- [5] Huerta-Cepas, J., Serra, F., & Bork, P. (2016). ETE 3: reconstruction, analysis, and visualization of phylogenomic data. *Molecular Biology and Evolution*, 33(6), 1635–1638.
- [6] Eaton, D. A. (2020). toytrees: A lightweight Python library for phylogenetics. *bioRxiv*.
- [7] Revell, L. B. (2012). phytools: An R package for phylogenetic comparative methods and other things. *Methods in Ecology and Evolution*, 3(2), 217–223.
- [8] Louca, S. (2018). Phylogenetic comparative methods on trees with known fossils. *Methods in Ecology and Evolution*.
- [9] Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1), 65–98.

A Detailed Benchmark Results

A.1 Tool Specifications

- *ape*: R 4.x, C backend
- *dendropy*: Python 3.x, pure Python
- *cogent3*: Python 3.x, optimized/compiled sections
- *Biopython*: Python 3.x, pure Python with wrappers
- *ete3*: Python 3.x, pure Python + compiled extensions (version-dependent)
- *toytree*: Python 3.x, Cython + C backend
- *Phylo.jl*: Julia 1.x, LLVM JIT compilation

A.2 Performance Plots

Plots showing traversal time distributions for each tool to be inserted here. Files: `ape_traverse.png`, `dendropy_traverse.png`, `cogent3_traverse.png`, `biopython_traverse.png`, `ete3_traverse.png`, `toytree_traverse.png`, `phylo_jl_traverse.png`

A.3 Raw Benchmark Data Tables

[Detailed tables with all timing measurements, statistical summaries, and system specifications to be included]